## UNIDAD I: INTRODUCCIÓN AL PROCESAMIENTO PARALELO

### 1.1.- Introducción: Procesamiento Paralelo – Más Allá de la Velocidad

El **procesamiento paralelo** no es solo una forma de que las computadoras trabajen más rápido; es una **arquitectura fundamental** que ha revolucionado la capacidad de cómputo y se ha vuelto indispensable en la era digital actual. En esencia, se refiere a la **ejecución simultánea de múltiples instrucciones o tareas** utilizando varios procesadores o núcleos, con el objetivo primordial de **reducir drásticamente el tiempo total de ejecución** de un programa o un conjunto de problemas.

Históricamente, el aumento de la velocidad de los ordenadores se lograba principalmente incrementando la **frecuencia del reloj** de los procesadores, es decir, la velocidad a la que realizan operaciones. Sin embargo, esta estrategia alcanzó rápidamente sus **límites físicos**. A medida que las frecuencias de reloj subían, también lo hacía la **generación de calor** en los chips, lo que hacía inviable seguir esta ruta sin enfrentar problemas de estabilidad, fiabilidad y consumo energético. Los procesadores de un solo núcleo, por muy rápidos que fueran, se encontraron con un techo de rendimiento.

Aquí es donde entra en juego el procesamiento paralelo, ofreciendo una solución elegante a estas limitaciones. En lugar de hacer que un solo procesador trabaje más rápido, la idea es que **múltiples procesadores trabajen en conjunto**. Para entenderlo mejor, imaginemos que tenemos que construir una pared: en lugar de que una sola persona la construya muy rápido, podemos tener a varias personas construyendo distintas secciones de la pared al mismo tiempo. Esta división del trabajo es el núcleo del paralelismo.

La **importancia crítica** del procesamiento paralelo se manifiesta en una amplia gama de campos tecnológicos y científicos que definen nuestra era:

- **Grandes Volúmenes de Datos (Big Data):** El procesamiento de bases de datos masivas, el análisis de tendencias en tiempo real, la minería de datos y la inteligencia de negocio serían imposibles sin arquitecturas paralelas. Imagina intentar procesar terabytes de información de transacciones bancarias o datos de redes sociales con un solo procesador; simplemente no se podría hacer en un tiempo útil.
- **Simulaciones Complejas:** Campos como la **meteorología** (modelos climáticos que predicen el tiempo), la **ingeniería** (simulaciones de estructuras, dinámica de fluidos, diseño de vehículos) y la **investigación científica** (modelado molecular, física de partículas) dependen de la capacidad de ejecutar billones de cálculos por segundo para simular fenómenos complejos. El paralelismo permite dividir estas simulaciones en partes más pequeñas que se resuelven concurrentemente.

- **Inteligencia Artificial (IA) y Machine Learning (ML):** El entrenamiento de modelos de IA, especialmente las redes neuronales profundas (Deep Learning), requiere una inmensa cantidad de operaciones matriciales y de vectorización que solo son factibles gracias a la capacidad de procesamiento paralelo masivo ofrecida por **Unidades de Procesamiento Gráfico (GPUs)** y otras arquitecturas especializadas.
- **Videojuegos y Realidad Virtual:** Para renderizar gráficos 3D complejos en tiempo real, simular físicas realistas y gestionar la lógica del juego, las consolas y las computadoras de gaming utilizan procesadores multinúcleo y GPUs con miles de núcleos paralelos. Esto crea experiencias inmersivas que serían imposibles con un enfoque secuencial.
- **Servidores Web y Computación en la Nube:** Los grandes centros de datos que alimentan Internet y los servicios en la nube dependen de miles de servidores trabajando en paralelo para manejar millones de solicitudes simultáneas, garantizando la disponibilidad y la escalabilidad de aplicaciones como Google, Facebook, Netflix y Spotify.

En resumen, el procesamiento paralelo no es una mejora opcional; es la **columna vertebral** de la computación moderna. Permite a los sistemas superar los **límites físicos** de los procesadores individuales, abordando la creciente demanda de velocidad y capacidad en un mundo impulsado por datos y aplicaciones cada vez más sofisticadas. Sin él, muchas de las tecnologías que damos por sentadas hoy simplemente no existirían o serían inviables.

Limitaciones Físicas que Impulsan el Paralelismo

- Ley de Moore y el Fin de la Escalabilidad en Frecuencia:

- Aumentar la frecuencia del reloj de un CPU single-core genera problemas de disipación térmica** (ej: chips que consumen >200W).

- Solución: Multiprocesadores paralelos (ej: CPUs de 64 núcleos) permiten mayor potencia sin sobrecalentamiento.

- Eficiencia Energética:

- El paralelismo permite dividir cargas de trabajo en núcleos de bajo voltaje, reduciendo el consumo (ej: arquitecturas ARM en móviles y servidores).

Desafíos del Procesamiento Paralelo

- Overhead de Comunicación: Coordinar tareas entre núcleos introduce latencia.

- Condiciones de Carrera y Sincronización: Múltiples hilos accediendo a recursos compartidos requieren mecanismos como *locks* o *semáforos*.

- Escalabilidad No Lineal: El *speedup* no siempre es proporcional al número de núcleos (ley de Amdahl).

El procesamiento paralelo no es solo una opción, sino el cimiento de la computación moderna. Desde impulsar la IA hasta modelar el clima, su aplicación trasciende la velocidad, permitiendo soluciones antes imposibles. Los futuros ingenieros deben dominar tanto sus principios teóricos (taxonomía de Flynn, modelos de consistencia) como sus herramientas prácticas (OpenMP, CUDA) para innovar en un mundo cada vez más dependiente de datos y simulaciones.

Recursos Adicionales:

- Video explicativo: "How Parallel Computing Works" (Computerphile).

- Herramientas para practicar: MPI (Message Passing Interface), NVIDIA CUDA Toolkit.

- Caso de estudio: Uso de paralelismo en el proyecto SETI@home para buscar vida extraterrestre.

### 1.2 Clasificación de las Arquitecturas y las Redes de Interconexión

Para comprender a fondo el procesamiento paralelo, es esencial clasificar cómo se organizan sus componentes. No todos los sistemas paralelos son iguales; varían en cómo los procesadores se conectan, acceden a la memoria y se comunican. Esta diversidad da lugar a diferentes modelos de arquitectura y redes de interconexión, cada uno con sus propias ventajas y desafíos.

#### 1.2.1 Clasificación de las Arquitecturas Paralelas: La Taxonomía de Flynn

La forma más influyente y ampliamente aceptada para clasificar las arquitecturas de computadores paralelas es la **Taxonomía de Flynn**, propuesta por Michael J. Flynn en 1972. Esta clasificación se basa en la multiplicidad de flujos de instrucciones y de datos. Analicemos cada una de sus cuatro categorías:

- **SISD (Single Instruction, Single Data - Una Instrucción, Un Dato):**
    - **Descripción:** Esta es la arquitectura de computadora secuencial tradicional. Un solo procesador ejecuta un único flujo de instrucciones sobre un único flujo de datos. No hay paralelismo a nivel de ejecución de instrucciones simultáneas.

    - **Ejemplos:** La mayoría de las computadoras personales antiguas y los microcontroladores simples operan bajo este principio. Incluso los procesadores modernos con múltiples núcleos siguen ejecutando hilos individuales en modo SISD dentro de cada núcleo.
    - **Relevancia:** Aunque no es paralela en sí misma, sirve como punto de referencia para entender las mejoras que ofrecen las otras categorías.
- **SIMD (Single Instruction, Multiple Data - Una Instrucción, Múltiples Datos):**
    - **Descripción:** Un solo procesador (o unidad de control) emite la misma instrucción a múltiples unidades de procesamiento, que operan simultáneamente sobre diferentes elementos de datos. Es ideal para tareas que requieren aplicar la misma operación repetidamente a grandes conjuntos de datos.
    - **Características:** Todas las unidades ejecutan la misma operación al mismo tiempo, pero cada una con su propio dato.
    - **Ejemplos:** Las **Unidades de Procesamiento Gráfico (GPUs)** son el ejemplo más prominente de arquitecturas SIMD. Cuando juegas un videojuego o entrenas un modelo de IA, la GPU aplica la misma operación de sombreado o cálculo a miles de píxeles o elementos de datos en paralelo. Otros ejemplos incluyen procesadores vectoriales y extensiones multimedia (como SSE o AVX en CPUs).
    - **Aplicaciones:** Procesamiento de imágenes y video, gráficos 3D, simulaciones científicas, criptografía y redes neuronales.
- **MISD (Multiple Instruction, Single Data - Múltiples Instrucciones, Un Dato):**
    - **Descripción:** Múltiples unidades de procesamiento ejecutan diferentes instrucciones simultáneamente sobre el mismo flujo de datos.
    - **Características:** Es la categoría menos común en arquitecturas de propósito general.
    - **Ejemplos:** Históricamente, se ha sugerido su uso en aplicaciones de tolerancia a fallos, donde múltiples procesadores ejecutan la misma tarea con instrucciones ligeramente diferentes sobre el mismo dato para verificar la consistencia. Algunos sistemas de control de vuelos espaciales podrían usar principios similares para redundancia.
    - **Relevancia:** Poco práctica para el aumento general del rendimiento, pero interesante para nichos muy específicos.

- **MIMD (Multiple Instruction, Multiple Data - Múltiples Instrucciones, Múltiples Datos):**
    - **Descripción:** Es la forma más flexible y potente de paralelismo. Múltiples procesadores ejecutan diferentes flujos de instrucciones de forma independiente sobre diferentes flujos de datos. Esto permite que cada procesador ejecute una tarea completamente diferente o una parte distinta de una tarea grande.
    - **Características:** Gran versatilidad y escalabilidad, pero también mayor complejidad en la gestión de la concurrencia y la comunicación.
    - **Ejemplos:** La mayoría de los sistemas paralelos modernos caen en esta categoría:
        - **Multiprocesadores con memoria compartida:** Computadoras con múltiples núcleos (los PCs modernos, servidores con muchos CPUs) donde todos los núcleos acceden a la misma memoria principal.
        - **Multicomputadores (o clústeres):** Grupos de computadoras independientes (nodos) conectadas por una red de alta velocidad, donde cada nodo tiene su propia memoria privada. Los superordenadores suelen ser grandes clústeres MIMD.
        - **Sistemas distribuidos:** Una forma aún más laxa de MIMD, donde computadoras geográficamente dispersas colaboran en una tarea.
    - **Aplicaciones:** Bases de datos distribuidas, computación en la nube, simulaciones a gran escala, inteligencia artificial distribuida y sistemas transaccionales.

### 1.2.2 Redes de Interconexión en Arquitecturas Paralelas

La forma en que los procesadores y la memoria se conectan entre sí es fundamental para el rendimiento de un sistema paralelo. La **red de interconexión** es el "camino" por donde viajan los datos y las instrucciones, y su diseño impacta directamente en la latencia, el ancho de banda y la escalabilidad del sistema.

Las redes de interconexión se pueden clasificar según su topología (la disposición de sus nodos y enlaces), su forma de conmutación (cómo se enrutan los mensajes) y su control (cómo se gestiona el tráfico).

**A. Topologías de Red:**

- **Directas (o Punto a Punto):** Los nodos están conectados directamente entre sí, formando una estructura específica.
    - **Anillo (Ring):** Los nodos están conectados en un círculo. Simple, pero si un nodo falla, el anillo se rompe, y la comunicación puede ser lenta para nodos distantes.
    - **Malla (Mesh):** Nodos organizados en una cuadrícula 2D (o 3D). Cada nodo se conecta a sus vecinos adyacentes. Permite múltiples rutas, pero la comunicación entre nodos muy separados puede requerir muchos saltos.
    - **Toro (Torus):** Una malla con conexiones adicionales en los bordes (los nodos de los extremos se conectan a los nodos opuestos), lo que reduce la distancia máxima entre nodos y mejora la redundancia.
    - **Hipercubo (Hypercube):** Una topología compleja donde el número de conexiones por nodo aumenta logarítmicamente con el número total de nodos. Ofrece muchas rutas cortas, pero es costosa de construir a gran escala.
    - **Árbol (Tree):** Los nodos se conectan de forma jerárquica. La comunicación es rápida si los datos fluyen hacia arriba o abajo en el árbol, pero el nodo raíz puede convertirse en un cuello de botella.
- **Indirectas (o con Conmutador/Switch):** Los nodos se conectan a través de un componente central o una red de conmutadores, no directamente entre sí.
    - **Bus:** Todos los procesadores comparten un único canal de comunicación. Simple y económico, pero el bus se convierte rápidamente en un **cuello de botella** de ancho de banda a medida que aumenta el número de procesadores. Muy común en sistemas multiprocesador de memoria compartida de pocos núcleos.
    - **Conmutador Cruzado (Crossbar Switch):** Permite que cualquier procesador se conecte directamente con cualquier módulo de memoria o con otro procesador sin conflictos, siempre que no intenten acceder al mismo destino simultáneamente. Ofrece alto ancho de banda, pero su complejidad y costo aumentan cuadráticamente con el número de nodos, haciéndolo inviable para grandes sistemas.
    - **Red Multietapa (Multistage Interconnection Network - MIN):** Consiste en múltiples etapas de pequeños conmutadores interconectados. Son un compromiso entre el bus (lento) y el crossbar (caro), ofreciendo escalabilidad y buen rendimiento. Ejemplos incluyen redes tipo Omega o Butterfly.

**B. Modos de Conmutación:**

- **Conmutación de Circuitos:** Se establece una ruta física dedicada entre origen y destino antes de que comience la transmisión de datos, similar a una llamada telefónica. Una vez establecido, el circuito es exclusivo.
- **Conmutación de Paquetes:** Los datos se dividen en pequeños "paquetes" independientes que se enrutan individualmente a través de la red. Cada paquete puede tomar una ruta diferente. Es más flexible y eficiente para tráfico irregular.
- **Conmutación de Wormhole:** Una optimización de la conmutación de paquetes donde los datos fluyen a través de la red tan pronto como llega la cabecera del paquete, sin esperar a que el paquete completo esté disponible en cada nodo. Reduce la latencia y el tamaño de los búferes necesarios en cada conmutador.

**C. Control de la Red:**

- **Centralizado:** Un único controlador gestiona todas las decisiones de enrutamiento y asignación de recursos. Simple de implementar, pero es un punto único de fallo y un cuello de botella para la escalabilidad.
- **Distribuido:** Las decisiones de enrutamiento y control se toman en cada nodo de forma independiente, basándose en la información local y en protocolos compartidos. Más complejo de diseñar, pero más robusto y escalable.

Comprender estas clasificaciones es fundamental, ya que la elección de una arquitectura y una red de interconexión adecuadas impacta directamente en el **rendimiento, el costo, la escalabilidad y la fiabilidad** de cualquier sistema de procesamiento paralelo.

---

### 1.3.- Paradigmas y Modelos de Programación Paralela

Una vez que hemos entendido cómo se clasifican las arquitecturas y cómo se interconectan sus componentes, el siguiente paso crucial es comprender **cómo podemos escribir programas que aprovechen este hardware paralelo**. Aquí es donde entran en juego los **paradigmas y modelos de programación paralela**, que nos proporcionan los marcos conceptuales y las herramientas prácticas para diseñar, implementar y gestionar la concurrencia. No existe un único "lenguaje" o "método" para la programación paralela; en cambio, existen diversas aproximaciones que se adaptan mejor a ciertos tipos de arquitecturas y problemas.

Un **paradigma de programación paralela** es un enfoque conceptual de alto nivel sobre cómo estructurar un programa para que sus tareas se ejecuten concurrentemente. Un **modelo de programación paralela** es una realización más concreta de un paradigma, a

menudo ligada a un conjunto específico de APIs (Interfaces de Programación de Aplicaciones), lenguajes o bibliotecas que permiten al programador expresar el paralelismo.

### 1.3.1. Paradigmas de Programación Paralela

Estos son los enfoques conceptuales principales para la paralelización:

- **Paralelismo de Datos (Data Parallelism):**
    - **Concepto:** Se enfoca en aplicar la **misma operación o conjunto de operaciones a diferentes elementos de un gran conjunto de datos** de forma simultánea. Las tareas son idénticas, pero los datos sobre los que operan son distintos.
    - **Ventajas:** Es un paradigma muy intuitivo para problemas con estructuras de datos regulares y operaciones uniformes. Es relativamente fácil de implementar en arquitecturas SIMD y es inherentemente escalable con el tamaño de los datos.
    - **Ejemplos:** Sumar dos vectores elemento a elemento, multiplicar matrices, aplicar un filtro a cada píxel de una imagen, entrenar una red neuronal donde cada neurona procesa una parte de los datos de entrada.
    - **Modelo de Programación Típico:** CUDA para GPUs, OpenMP (para bucles paralelos), MPI (para distribuir arrays).
- **Paralelismo de Tareas (Task Parallelism / Function Parallelism):**
    - **Concepto:** Implica la **ejecución concurrente de diferentes tareas o funciones** que pueden o no operar sobre los mismos datos. Cada tarea es una unidad de trabajo independiente o semi-independiente.
    - **Ventajas:** Ofrece gran flexibilidad para paralelizar programas heterogéneos, donde distintas partes del código realizan funciones diversas. Es adecuado para la mayoría de las arquitecturas MIMD.
    - **Ejemplos:** Un servidor web que maneja múltiples solicitudes de clientes simultáneamente (cada solicitud es una tarea), un pipeline de procesamiento de datos donde una etapa filtra, otra transforma y otra almacena los datos, o un sistema operativo que ejecuta múltiples aplicaciones al mismo tiempo.
    - **Modelo de Programación Típico:** Threads (Pthreads, C++ std::thread), OpenMP (para secciones paralelas), modelos de paso de mensajes como MPI.

- **Paralelismo de Pipeline (Pipeline Parallelism):**
    - **Concepto:** Similar a una línea de ensamblaje. Una tarea se descompone en una serie de etapas secuenciales, y cada etapa se ejecuta por un procesador diferente. A medida que una etapa termina de procesar un dato, pasa el resultado a la siguiente etapa, y al mismo tiempo comienza a procesar un nuevo dato.
    - **Ventajas:** Permite una alta tasa de procesamiento continuo de datos (throughput), ya que varias piezas de datos están en diferentes etapas de procesamiento simultáneamente.
    - **Ejemplos:** Las etapas de ejecución de instrucciones en una CPU (fetch, decode, execute, write-back), un sistema de renderizado de gráficos donde una etapa procesa vértices, otra texturas y otra la rasterización, o un sistema de procesamiento de audio en tiempo real.
    - **Modelo de Programación Típico:** Implementaciones a medida con hilos o procesos que se comunican de forma secuencial.

### 1.3.2. Modelos de Programación Paralela

Estos modelos son las herramientas y API que usamos para implementar los paradigmas:

- **Modelos de Memoria Compartida (Shared Memory Models):**
    - **Concepto:** Se asumen arquitecturas donde todos los procesadores pueden acceder directamente a un espacio de memoria común. La comunicación entre tareas se realiza leyendo y escribiendo en ubicaciones de memoria compartidas.
    - **Ventajas:** Relativamente fácil de programar para problemas que requieren acceso frecuente a datos comunes. La comunicación es implícita y rápida, ya que no se necesita enviar datos explícitamente entre procesadores.
    - **Desafíos:**
        - **Sincronización:** Requiere mecanismos como **locks (candados), mutexes (exclusión mutua), semáforos** y **barreras** para evitar **condiciones de carrera** (cuando múltiples hilos intentan acceder o modificar el mismo dato compartido al mismo tiempo, llevando a resultados impredecibles) y garantizar la **coherencia de caché**.

        - **Escalabilidad:** Puede ser difícil escalar a un gran número de procesadores debido a la contención por el acceso a la memoria y los problemas de coherencia.
    - **Ejemplos de APIs/Librerías:**
        - **OpenMP (Open Multi-Processing):** Un conjunto de directivas de compilador, rutinas de biblioteca y variables de entorno que permiten la programación paralela de memoria compartida en lenguajes como C, C++ y Fortran. Es muy utilizado para paralelizar bucles y secciones de código.
        - **Pthreads (POSIX Threads):** Una API estándar de bajo nivel para crear y gestionar hilos (threads) en sistemas operativos compatibles con POSIX (como Linux). Ofrece un control granular sobre los hilos y la sincronización.
        - **Java Threads, C++ std::thread:** Abstracciones de hilos nativas en lenguajes de programación de alto nivel.
- **Modelos de Paso de Mensajes (Message Passing Models):**
    - **Concepto:** Se asumen arquitecturas donde cada procesador tiene su propia memoria privada. La comunicación entre procesadores se realiza enviando y recibiendo mensajes explícitamente a través de la red de interconexión.
    - **Ventajas:**
        - **Escalabilidad:** Ideal para arquitecturas distribuidas (clústeres, multicomputadores) con un gran número de nodos, ya que cada nodo opera con su propia memoria local, minimizando la contención.
        - **Menos problemas de coherencia de caché:** Al no haber memoria compartida directa, los problemas de coherencia de caché son menos prevalentes a nivel de programador (aunque la red de interconexión debe garantizar la entrega de mensajes).
    - **Desafíos:**
        - **Programación más compleja:** Requiere que el programador gestione explícitamente la comunicación y la distribución de datos.

        - **Sobrecarga de comunicación:** El envío y la recepción de mensajes pueden introducir latencia significativa si no se optimiza.
    - **Ejemplos de APIs/Librerías:**
        - **MPI (Message Passing Interface):** El estándar de facto para la programación de paso de mensajes en sistemas distribuidos y supercomputadores. Proporciona funciones para enviar, recibir, realizar operaciones colectivas (broadcast, reduce) y gestionar la topología de procesos. Es independiente del lenguaje (C, C++, Fortran).
- **Modelos Basados en Hardware Especializado (Accelerators/Manycore):**
    - **Concepto:** Utilizan arquitecturas con un gran número de núcleos de procesamiento, a menudo más simples y especializados que los núcleos de CPU tradicionales, diseñados para un alto paralelismo de datos.
    - **Ventajas:** Rendimiento masivo para tareas altamente paralelas, eficiencia energética.
    - **Desafíos:** Requiere modelos de programación específicos y, a menudo, una reorganización significativa del algoritmo para adaptarse a la arquitectura.
    - **Ejemplos de APIs/Librerías:**
        - **CUDA (Compute Unified Device Architecture):** Desarrollado por NVIDIA para programar sus GPUs. Permite a los desarrolladores utilizar C, C++ o Fortran con extensiones específicas para lanzar kernels (funciones paralelas) en la GPU.
        - **OpenCL (Open Computing Language):** Un estándar abierto para la programación heterogénea que permite escribir programas que se ejecutan en GPUs, CPUs y otros aceleradores de diferentes fabricantes.
- **Modelos de Programación de Alto Nivel (High-Level Parallel Programming):**
    - **Concepto:** Abstracciones que intentan simplificar la programación paralela ocultando muchos de los detalles de bajo nivel. El sistema runtime o el compilador se encarga de la distribución de tareas, la comunicación y la sincronización.
    - **Ventajas:** Mayor productividad del programador, menos errores relacionados con la concurrencia, portabilidad.

- **Desafíos:** Menor control sobre la optimización fina del rendimiento, puede que no sea adecuado para todos los tipos de problemas.
- **Ejemplos:**
  - **MapReduce (Hadoop, Spark):** Un modelo popular para el procesamiento de grandes volúmenes de datos en clústeres, donde las operaciones se expresan como fases de "map" (transformación) y "reduce" (agregación).
  - **Lenguajes de programación funcional:** Algunos lenguajes funcionales (como Haskell o Erlang) tienen propiedades que facilitan el paralelismo implícito o concurrencia sin estado compartido.
  - **Frameworks de paralelismo de tareas (TBB, Cilk Plus, etc.):** Bibliotecas que proporcionan construcciones para expresar paralelismo a nivel de tareas de forma más sencilla que los hilos crudos.

La elección del paradigma y el modelo de programación depende críticamente de la **arquitectura paralela subyacente**, la **naturaleza del problema a resolver**, y los **requisitos de rendimiento y complejidad** del desarrollo. Un buen diseño de software paralelo comienza con la selección de la aproximación adecuada.

---

### 1.4.- Medidas de Rendimiento, Análisis de la Escalabilidad y el *Speedup*

Para determinar si un sistema paralelo es efectivo y para optimizar su desempeño, necesitamos herramientas para **medir y analizar su rendimiento**. No basta con que un programa se ejecute en paralelo; debemos cuantificar cuánto mejor lo hace y bajo qué condiciones. Las **medidas de rendimiento** nos permiten comparar diferentes arquitecturas, algoritmos y configuraciones. De la mano con esto, el análisis de la **escalabilidad** y el **speedup** nos ayuda a entender cómo un sistema se comporta a medida que aumentamos los recursos computacionales.

#### 1.4.1. Medidas Básicas de Rendimiento

Antes de profundizar en el paralelismo, es importante recordar las métricas fundamentales para evaluar el desempeño de cualquier sistema computacional:

- **Tiempo de Ejecución (Execution Time):** Es la medida más directa. Se refiere al tiempo transcurrido desde que un programa comienza hasta que finaliza. En el contexto del paralelismo, distinguimos:

    - **Tiempo Secuencial (Ts):** El tiempo que le toma al mejor algoritmo secuencial conocido ejecutar la tarea en un único procesador.
    - **Tiempo Paralelo (Tp):** El tiempo que le toma al algoritmo paralelo ejecutar la tarea utilizando P procesadores.
    - *Importancia:* Minimizar Tp es el objetivo principal del procesamiento paralelo.
- **Rendimiento (Throughput):** Es la cantidad de trabajo que un sistema puede completar en una unidad de tiempo. Por ejemplo, el número de transacciones procesadas por segundo, o el número de imágenes renderizadas por minuto.
    - *Importancia:* Crucial en sistemas donde el volumen de trabajo continuo es clave, como servidores web o sistemas de procesamiento de datos en tiempo real.
- **Latencia (Latency):** Es el tiempo que tarda una operación individual en completarse, o el tiempo que tarda un mensaje en viajar de un punto a otro.
    - *Importancia:* Reducir la latencia es vital en aplicaciones interactivas o donde la comunicación es frecuente entre procesadores.

### 1.4.2. *Speedup* (Aceleración)

El **speedup** es la métrica fundamental para cuantificar la **ganancia de rendimiento** de una implementación paralela sobre una secuencial. Nos dice cuántas veces más rápido se ejecuta el programa cuando se utilizan múltiples procesadores.

La fórmula es:

S(P)=Tiempo de ejecucio´n del algoritmo paralelo con P procesadores (Tp )Tiempo de ejecucio´n del mejor algoritmo secuencial (Ts)

- **Interpretación:**
    - Si S(P)=1, no hay ganancia de rendimiento (el paralelo es igual de rápido que el secuencial).
    - Si S(P)<1, el programa paralelo es más lento que el secuencial (lo que indica que la paralelización no es efectiva o introduce demasiada sobrecarga).
    - Si S(P)>1, hay una ganancia de rendimiento.
    - El **speedup ideal** (o *speedup* lineal) es cuando S(P)=P. Esto significa que duplicar el número de procesadores duplica la velocidad de ejecución. Esto es raro de lograr en la práctica debido a la sobrecarga de paralelización.

- **Factores que afectan el *speedup*:**
    - **Porción secuencial del código:** Incluso un pequeño porcentaje de código que no puede paralelizarse limitará severamente el *speedup* total.
    - **Sobrecarga de comunicación:** El tiempo que los procesadores dedican a enviarse mensajes o a sincronizarse entre sí resta tiempo de computación útil.
    - **Desequilibrio de carga:** Si algunas tareas son más grandes que otras y no se distribuyen uniformemente, algunos procesadores estarán ociosos esperando que otros terminen.
    - **Contención de recursos:** Múltiples procesadores compitiendo por el acceso a una memoria compartida o un bus.

### 1.4.3. Eficiencia

La **eficiencia** nos indica **qué tan bien se están utilizando los procesadores adicionales** para lograr el *speedup*. Complementa el *speedup* al proporcionar una perspectiva sobre el costo-beneficio de añadir más recursos.

La fórmula es:

$$E(P) = \frac{S(P)}{P} = \frac{\text{Tiempo de ejecución paralelo con P procesadores}}{\text{Tiempo de ejecución secuencial}}$$

- **Interpretación:**
    - La eficiencia siempre estará entre 0 y 1 (o 0% y 100%).
    - Una eficiencia de $E(P)=1$ (o 100%) indica un uso perfecto de todos los procesadores; cada procesador contribuye plenamente a la aceleración.
    - A medida que P aumenta, la eficiencia tiende a disminuir debido a la sobrecarga de paralelización. Una eficiencia baja sugiere que se están desperdiciando recursos.
    - *Importancia:* Una alta eficiencia es crucial para la justificación económica de sistemas paralelos, ya que indica un buen retorno de la inversión en hardware.

### 1.4.4. Análisis de la Escalabilidad

La **escalabilidad** de un sistema paralelo se refiere a su capacidad para mantener un buen rendimiento (o una alta eficiencia) a medida que aumentamos el número de procesadores (P) o el tamaño del problema. Un sistema es "altamente escalable" si su *speedup* sigue creciendo de forma casi lineal incluso con muchos procesadores.

Existen dos enfoques principales para analizar la escalabilidad:

- **Escalabilidad Fija del Problema (Strong Scaling):**
    - **Concepto:** El **tamaño total del problema se mantiene constante**, y se observa cómo el tiempo de ejecución disminuye a medida que se aumenta el número de procesadores.
    - **Objetivo:** Reducir el tiempo total de ejecución para un problema dado.
    - **Desafío:** La Ley de Amdahl es el principal limitante aquí. A medida que P aumenta, la porción secuencial del problema domina el tiempo de ejecución, y el *speedup* se estanca.
    - **Ejemplo:** Resolver una ecuación diferencial específica en menos tiempo añadiendo más CPUs.
- **Escalabilidad Fija del Procesador (Weak Scaling):**
    - **Concepto:** El **tamaño del problema por procesador se mantiene constante**, lo que significa que el tamaño total del problema crece proporcionalmente con el número de procesadores.
    - **Objetivo:** Mantener el tiempo de ejecución (o la eficiencia) constante a medida que se escala el problema y los recursos.
    - **Ventaja:** Permite resolver problemas mucho más grandes manteniendo una eficiencia razonable, ya que la sobrecarga de comunicación y la porción secuencial a menudo son una fracción constante del trabajo por procesador. La Ley de Gustafson-Barsis es más relevante aquí.
    - Ley de Gustafson-Barsis: $S(P)=P-(P-1)s$

Donde s es la fracción del tiempo de ejecución total que es secuencial para un problema escalado. Demuestra que si el tamaño del problema se escala con P, se puede lograr un speedup casi lineal.

- **Ejemplo:** Procesar una base de datos más grande distribuyéndola entre más nodos, manteniendo el tiempo de procesamiento por nodo similar al de una base de datos más pequeña en un solo nodo.

### 1.4.5. Cuellos de Botella y su Identificación

Un **cuello de botella** es cualquier componente o fase de un sistema que restringe el rendimiento general, limitando la capacidad de la aplicación o del sistema para escalar. Identificar y eliminar cuellos de botella es crucial para mejorar el *speedup* y la eficiencia.

- **Fuentes Comunes de Cuellos de Botella:**

- **Comunicación:** Si los procesadores pasan demasiado tiempo enviando y recibiendo datos entre sí (alta latencia de mensajes, bajo ancho de banda de la red).
- **Sincronización:** Operaciones de sincronización excesivas (bloqueos, barreras) que obligan a los procesadores a esperar unos a otros.
- **Acceso a memoria:** Contención por el acceso a la memoria compartida, o fallos de caché frecuentes.
- **Porción secuencial:** Como vimos con la Ley de Amdahl, la parte no paralelizable del código es un cuello de botella inherente.
- **Desequilibrio de carga:** Algunos procesadores terminan sus tareas antes que otros y quedan inactivos.
- **E/S (Entrada/Salida):** Si la aplicación está esperando continuamente datos del disco o de la red.

- **Técnicas de Identificación:**
  - **Herramientas de Perfilado (Profilers):** Software que mide el tiempo que un programa pasa en diferentes secciones de código o realizando diferentes operaciones (ej. valgrind, gprof, Intel VTune, herramientas específicas para MPI/OpenMP).
  - **Análisis de Trazas:** Registrando eventos (envío/recepción de mensajes, entradas/salidas de barreras) y visualizándolos para identificar patrones de inactividad o comunicación ineficiente.
  - **Simulación:** Usar simuladores (como los que se verán en las unidades siguientes) para modelar el comportamiento del sistema bajo diferentes condiciones y topologías.
  - **Monitoreo del Sistema:** Observar el uso de CPU, memoria, red y disco durante la ejecución.

Al dominar estas medidas y técnicas de análisis, los ingenieros informáticos pueden diseñar y optimizar sistemas paralelos que no solo sean rápidos, sino también eficientes y escalables, maximizando el uso de los recursos computacionales disponibles.